Authentication and Authorization Report

Verdant — IoT Environmental Monitoring System

Laravel 13 Backend + Vue 3 Dashboard + ESP Sensor Nodes

Name	Student ID
Raffael Valentino Gunawan	00000091170
Muhammad Sagara Waluya	00000101478

Demonstration Video

A video walkthrough demonstrating the authentication and authorization system in action is available at:
https://youtu.be/LLTRRBKQZA4

1 Introduction

1. **User authentication** using email/password and Google OAuth via **Laravel Sanctum** SPA session authentication.
2. **Device authentication** using a stateless API token sent in the `X-Device-Token` HTTP header.

User authenticatio

2 Entities Being Authenticated

The system authenticates two distinct classes of entities:

2.1 Users

The `User` model (`app/Models/User.php`) extends Laravel's `Authenticatable` and stores:

- `name`, `email`, `password` (hashed via bcrypt)
- `google_id` (for Google OAuth linking)
- `avatar` (OAuth profile picture URL)
- `email_verified_at`

Users own plants. Each plant record has a `user_id` foreign key, and authorization is scoped to plant ownership (see Section 5).

2.2 Devices

The `Device` model (`app/Models/Device.php`) stores:

- `name`, `serial_number`
- `api_token` — the SHA-256 hash of the device's plaintext API token
- `firmware_version`

- `status` — an enum: `active`, `inactive`, or `maintenance`
- `last_seen_at` — updated on every authenticated reading ingest

Devices belong to a user (via `user_id`) and may be assigned to a plant (via `plant_id`).

3 Authentication

3.1 User Authentication: Laravel Sanctum SPA

Users authenticate to the Vue dashboard through **Laravel Sanctum**. Sanctum is Laravel's official package for single-page application (SPA) authentication. Instead of issuing bearer tokens stored in `localStorage`, Sanctum issues **HttpOnly session cookies** that are automatically sent by the browser with every request to the same origin.

3.1.1 Inner workings

The authentication flow is:

1. The frontend calls `GET /sanctum/csrf-cookie`. Laravel sets two cookies:
   - `laravel_session` (HttpOnly) — the session identifier
   - `XSRF-TOKEN` — a CSRF token readable by JavaScript
2. The frontend posts the user's email and password to `POST /api/login`.
3. Laravel verifies the credentials against the `users` table using the `web` guard (session driver, Eloquent provider).
4. On success, Laravel calls `$request->session()->regenerate()` to prevent session fixation, then issues the session cookie.
5. Every subsequent API request from the same origin automatically includes the session cookie. Sanctum's `auth:sanctum` guard validates the session and resolves the authenticated user.
6. On logout, the session is invalidated and regenerated.

3.1.2 CSRF protection

State-changing requests (`POST`, `PUT`, `DELETE`, `PATCH`) require the `X-XSRF-TOKEN` header to match the `XSRF-TOKEN` cookie. This prevents cross-site request forgery attacks. Laravel's `ValidateCsrfToken` middleware enforces this. The frontend's axios instance is configured with `withXSRFToken: true` so it copies the cookie value into the header automatically.

3.1.3 Configuration

The Sanctum configuration (`config/sanctum.php`) specifies:

- `stateful` domains — origins allowed to receive stateful session cookies (`localhost:5173` in development)
- `guard` — `['web']`, the session-based guard
- `expiration` — `null` (session expires when the browser closes, consistent with cookie-based SPA auth)

Rate limiting on login is handled by a `throttle:login` middleware (5 attempts per minute) defined in `bootstrap/app.php`.

3.2 User Authentication: Google OAuth

Users may also authenticate via **Google OAuth 2.0** using Laravel Socialite. The flow is:

1. The user clicks "Continue with Google" on the login page.
2. The browser is redirected to Google's consent screen (`GET /api/auth/google`).
3. After the user consents, Google redirects back to `GET /api/auth/google/callback` with an authorization code.
4. The backend exchanges the code for a Google profile (name, email, avatar, Google ID) via Socialite.
5. The backend looks up or creates a `User` record keyed by `google_id` or `email`.
6. Laravel logs the user in (`Auth::guard('web')->login()`), issuing the same Sanctum session cookie as email/password login.
7. The browser is redirected back to the frontend dashboard.

3.3 Device Authentication: Stateless API Token

Devices authenticate using a stateless API token. No session, no cookies, no CSRF. The token is sent in the `X-Device-Token` HTTP header on every request.

3.3.1 Inner workings

The `AuthenticateDevice` middleware (`app/Http/Middleware/AuthenticateDevice.php`) handles device authentication:

1. Read the `X-Device-Token` header from the request.
2. Hash the presented token with SHA-256.
3. Look up the device by comparing against the stored `api_token` column (which holds `sha256(token)`).
4. If no device matches, return `401 Unauthorized`.
5. If the device's `status` is not `active`, return `403 Forbidden`.
6. Update `last_seen_at` to mark the device as online.
7. Attach the resolved `Device` model to the request attributes so the controller can use it without a second lookup.

3.3.2 Why hash the token?

Storing `sha256(token)` instead of the plaintext token means that even if the database is compromised, an attacker cannot recover the original device tokens and impersonate devices. The plaintext token exists only at generation time (`Device::generateToken()`) and is handed to whoever provisions the device (written to `storage/app/private/device-tokens.json` by the seeder, or shown once when a plant is created with a device token).

Listing 1: Token generation in Device model

```
$token = Str::random(40);
$this->api_token = hash('sha256', $token);
$this->save();
return $token; // returned once, never recoverable afterwards
```

3.3.3 Configuration

The `device.auth` middleware alias is registered in `bootstrap/app.php`:

Listing 2: Middleware alias registration

```
$middleware->alias([
    'device.auth' => AuthenticateDevice::class,
]);
```

To ensure the device is resolved before rate limiting, `ThrottleRequests` is prepended to the priority list after `AuthenticateDevice`:

Listing 3: Middleware priority ordering

```
$middleware->prependToPriorityList(
    ThrottleRequests::class,
    AuthenticateDevice::class
);
```

This guarantees the ingest rate limiter keys on the **device** (not the IP address), so multiple devices behind the same NAT are rate-limited independently.

3.4 Rate Limiting

Two rate limiters protect the system:

Limiter	Applies to	Limit
`throttle:login`	`POST /api/login`	5 attempts / minute
`throttle:device-ingest`	`POST /api/readings`	30 requests / minute per device

Table 1: Rate limiters

4 Roles and Access Levels

The system does not define traditional named roles (e.g., "admin," "editor"). Instead, access is determined by **entity type** and **ownership**:

Entity	Authentication	Access level
User (dashboard)	Sanctum session cookie (email/password or Google OAuth)	Can manage only plants they own (`user_id` match). Other users' plants return `404`.
Device	`X-Device-Token` header (stateless, SHA-256)	Can POST readings only for its own device. Token = device identity.

Table 2: Authentication entities and access levels

Device status (`DeviceStatus` enum) acts as an access gate:

- `active` — device may submit readings
- `inactive` / `maintenance` — device is rejected with `403 Forbidden`

There is no separate device "role" beyond its active/inactive status and its association with a user and plant.

5 Authorization

Authorization answers the question "what may this authenticated entity do?"

5.1 User Authorization: PlantPolicy

Plant ownership is enforced through a Laravel **Policy** class (`app/Policies/PlantPolicy.php`). A plant "belongs to exactly one user," and everything hanging off it (devices, readings, thresholds, alerts) inherits that ownership.

Listing 4: PlantPolicy authorization rules

```
public function view(User $user, Plant $plant): Response
{
    return $this->owns($user, $plant);
}

public function update(User $user, Plant $plant): Response
{
    return $this->owns($user, $plant);
}

public function delete(User $user, Plant $plant): Response
{
    return $this->owns($user, $plant);
}

private function owns(User $user, Plant $plant): Response
{
    return $user->id === $plant->user_id
        ? Response::allow()
        : Response::denyAsNotFound();
}
```

Key design choice: `denyAsNotFound()` returns an HTTP `404 Not Found` instead of `403 Forbidden` when a user accesses another user's plant. This is a deliberate **security-by-obscurity** measure: the API does not confirm whether a given plant ID exists if the requesting user does not own it, preventing enumeration of other users' plants.

The controller enforces authorization via Laravel's `Gate`:

Listing 5: Gate::authorize in PlantController

```
public function show(Plant $plant): PlantResource
{
    Gate::authorize('view', $plant);
    return new PlantResource($plant->load(...));
}

public function update(PlantRequest $request, Plant $plant):
   PlantResource
{
    Gate::authorize('update', $plant);
    $plant->update($request->validated());
    ...
}

public function destroy(Plant $plant): Response
{
    Gate::authorize('delete', $plant);
```

```
    $plant->delete();
    ...
}
```

The plant list endpoint (`index`) does not need an explicit gate because it queries only the authenticated user's own plants:

Listing 6: User-scoped plant list

```
$plants = $request->user()
    ->plants()
    ->with(...)
    ->orderBy('name')
    ->get();
```

5.2 Device Authorization

Device authorization is implicit in authentication: the `X-Device-Token` identifies a specific device, and that device may only POST readings to its own ingest endpoint (`POST /api/readings`). The resolved device is attached to the request, and the `ReadingController` associates the reading with that device. A device cannot read or modify another device's data because the ingest endpoint is write-only for devices.

5.3 Route-Level Authorization

All dashboard API routes are grouped under the `auth:sanctum` middleware, meaning unauthenticated requests are rejected with `401` before reaching any controller:

Listing 7: Route group with sanctum middleware

```
Route::middleware('auth:sanctum')->group(function () {
    Route::post('/logout', ...);
    Route::get('/user', ...);
    Route::apiResource('plants', PlantController::class);
    Route::put('/devices/{device}/thresholds', ...);
    Route::get('/alerts', ...);
});
```

Device routes use the `device.auth` middleware instead:

Listing 8: Device route with device.auth middleware

```
Route::post('/readings', [ReadingController::class, 'store'])
    ->middleware(['device.auth', 'throttle:device-ingest']);
```

5.4 Frontend Route Guards

On the frontend, the Vue router (`frontend/src/router.js`) implements client-side guards:

- Routes with `meta: { guest: true }` (e.g., `/login`) redirect authenticated users to the dashboard.
- Routes without `guest` redirect unauthenticated users to `/login`.
- A global `onUnauthorized` handler listens for `401` responses from the API and automatically logs the user out and redirects to the login page.

These are convenience guards for the user experience; the backend remains the source of truth for authorization.

6 Security Summary

Control	Implementation
Password storage	bcrypt hashing (Laravel `Hidden` on `password` field)
Session fixation prevention	`session()->regenerate()` on login and logout
Token storage	`sha256(token)` stored; plaintext never persisted
Session type	HttpOnly cookie (inaccessible to JavaScript, mitigates XSS token theft)
CSRF protection	`XSRF-TOKEN` cookie + `X-XSRF-TOKEN` header on state-changing requests
Authorization bypass	Plant ownership check via `PlantPolicy`; `denyAsNotFound()` prevents ID enumeration
Device lockout	`status` must be `active`; `inactive`/`maintenance` returns `403`
Rate limiting	5 login attempts/min; 30 device ingest requests/min per device
OAuth	Google OAuth 2.0 via Laravel Socialite; stateless driver

Table 3: Security controls summary

7 Demo Accounts

The seeder creates two demo accounts (password: `password`):

Email	Plants	Purpose
`test@example.com`	Monstera, Cactus, Fiddle Leaf Fig	Primary user account
`second@example.com`	Calathea, Boston Fern	Second user (for cross-user authorization testing)

Table 4: Demo accounts created by `php artisan migrate --seed`

Device tokens are written to `backend/storage/app/private/device-tokens.json` during seeding.

8 Conclusion

Verdant implements a dual-guard authentication and authorization architecture: Laravel Sanctum session-based SPA authentication for human users of the dashboard, and stateless SHA-256-hashed API token authentication for ESP sensor devices. Authorization is ownership-based, enforced through Laravel Policies and Gates, with deliberate 404 responses to prevent resource enumeration. Rate limiting, CSRF protection, session fixation prevention, and HttpOnly cookies round out the defense-in-depth strategy.